Name :- Yug Kapuriya

UNID :- UMID13012677975

Gmail :- yugkapuriya404@gmail.com

LinkedIn :- Yug Kapuriya

Password Cracking & Credential Attack Suite

Technical Documentation

A practical educational toolkit for password policy testing

Ethical Use Only | Authorized Environments Only

Modules Included

1. Dictionary Generator
2. Hash Extractor
3. Brute Force Simulator
4. Password Analyzer
5. Report Generator

PROJECT OVERVIEW

Purpose

- Understand how passwords are stored and hashed
- Simulate dictionary and brute-force attacks in a safe environment
- Analyze password strength using entropy and complexity rules
- Generate audit reports with security recommendations

Tools and Technologies

Technology	Purpose
Python 3.x	Core programming language
itertools	Brute-force character combinations
hashlib	MD5, SHA-1, SHA-256, SHA-512 hashing
math	Entropy calculation
platform / subprocess	OS detection and hash extraction

System Architecture

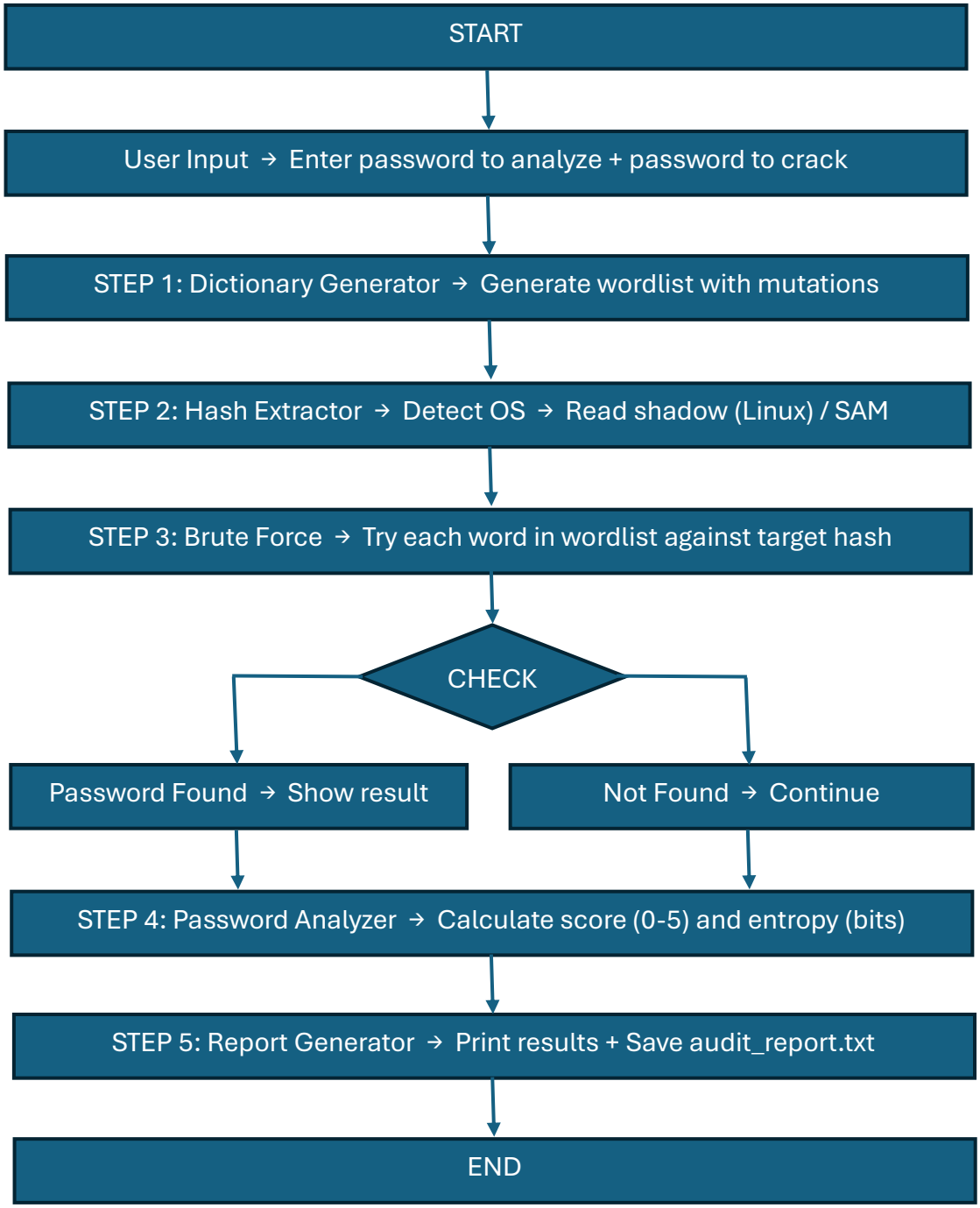
Module Structure

File	Module	Function
main.py	Automation	Calls all modules in order
dictionary_generator.py	Dictionary Generator	Generates password wordlist
hash_extractor.py	Hash Extractor	Extracts and identifies hashes
brute_force.py	Brute Force Simulator	Cracks passwords via iteration
password_analyzer.py	Password Analyzer	Scores strength and entropy
report_generator.py	Report Generator	Generates audit_report.txt

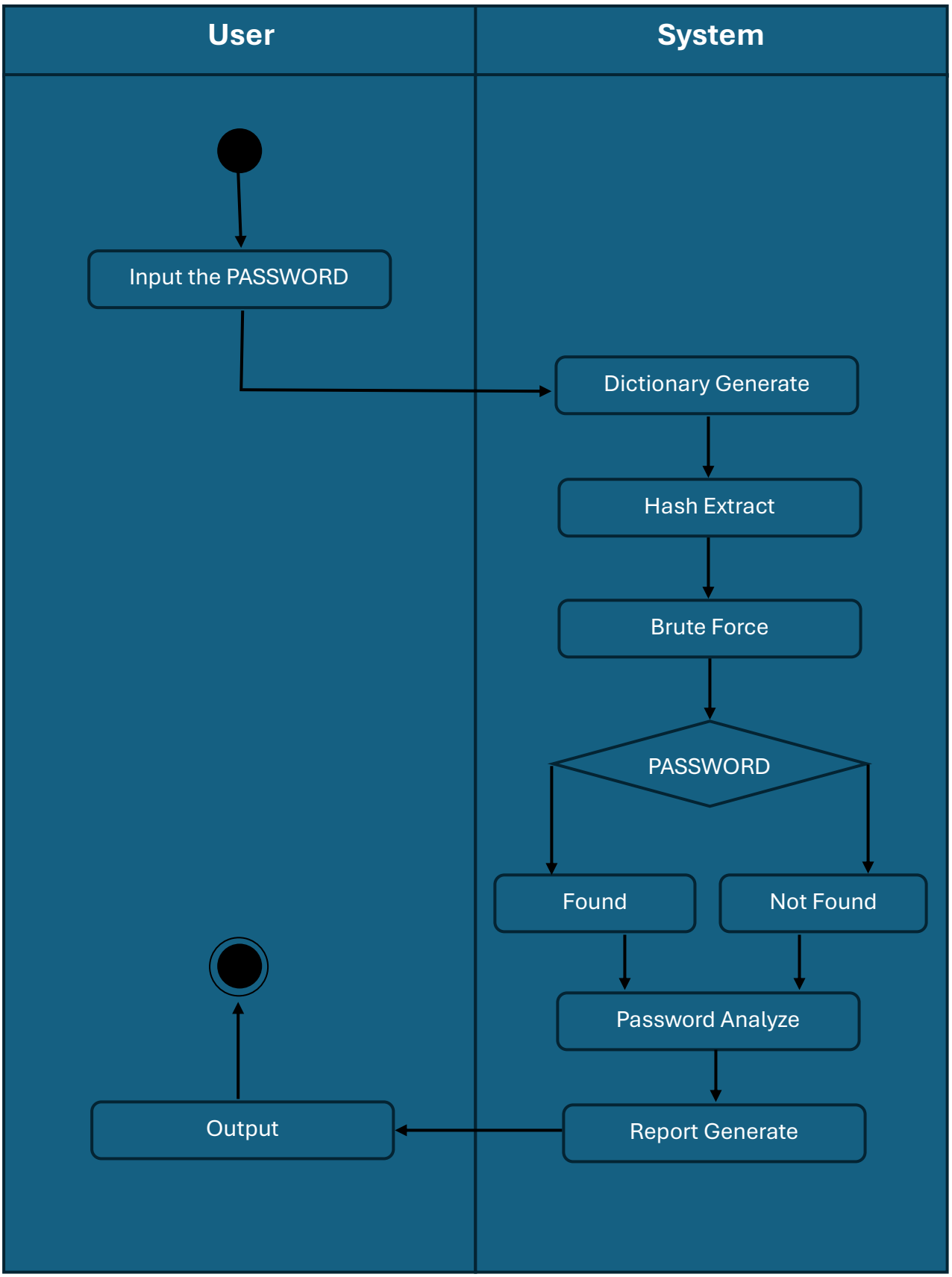
Diagram

System Flow Chart

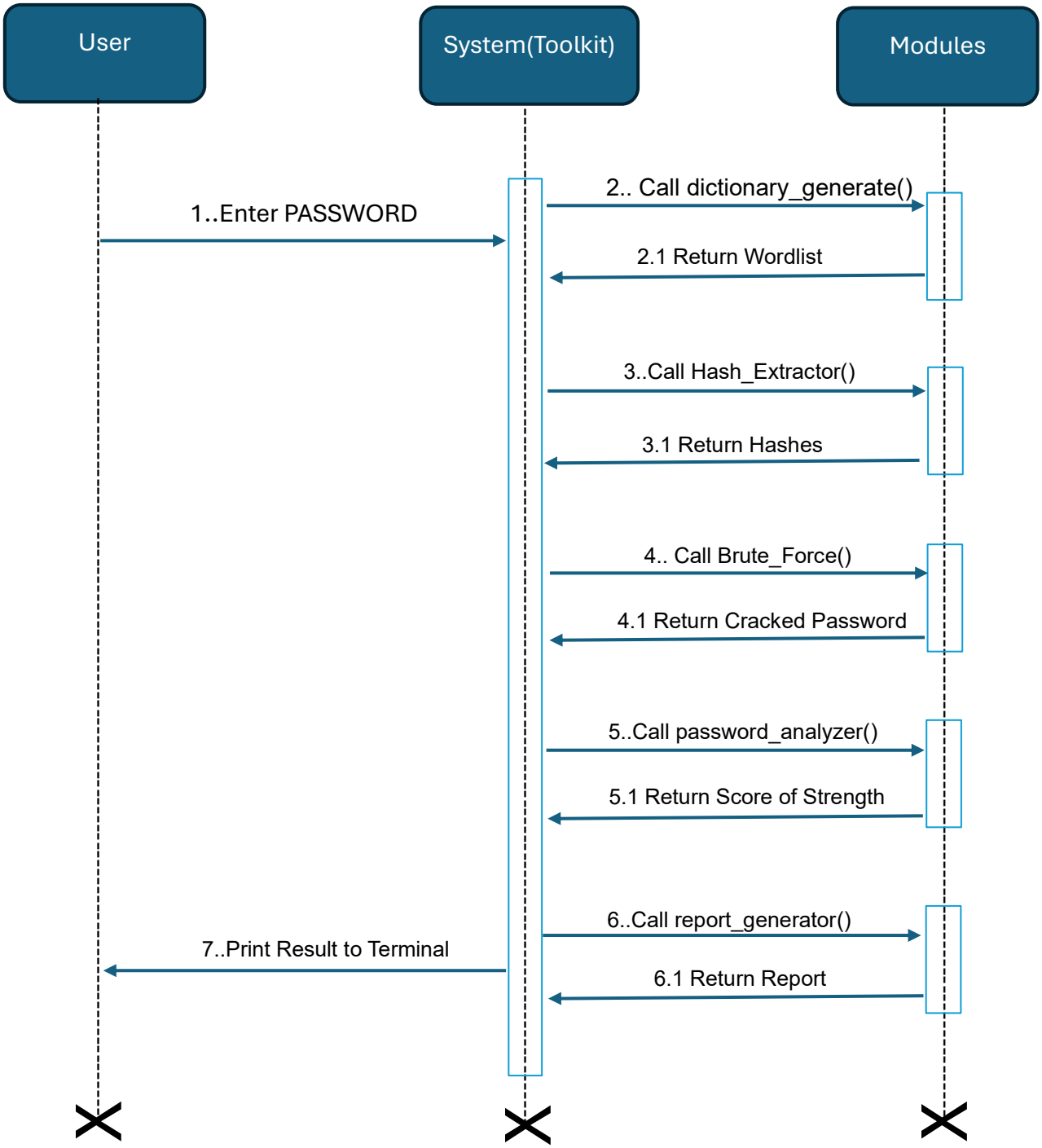
The flowchart below shows the complete execution flow from user input to final report.



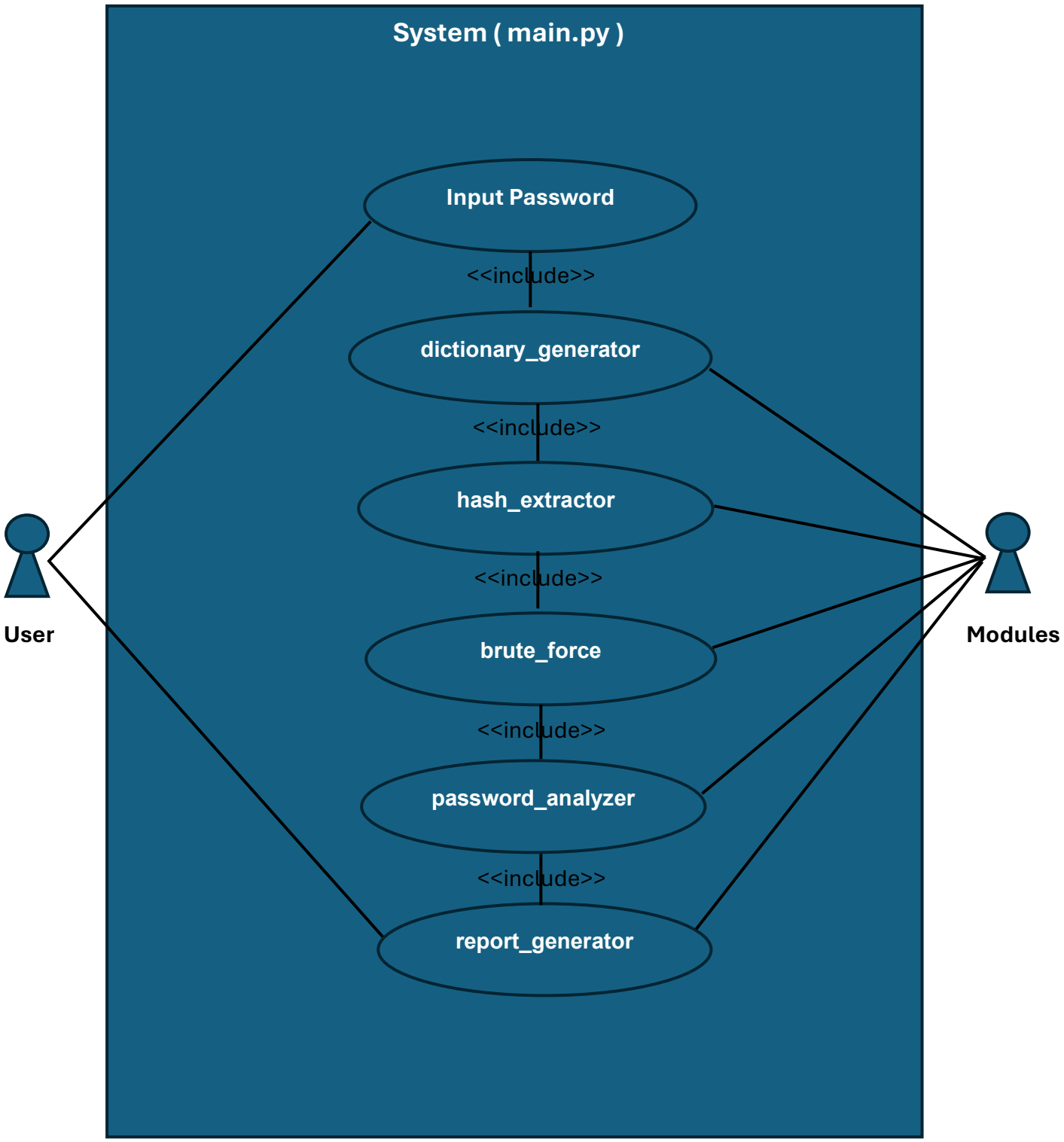
Activity Diagram



Sequence Diagram



Use Case Diagram



Module Descriptions

Module 1 — Dictionary Generator

Generates a custom wordlist of password candidates based on common names, years, symbols, and leet-speak transformations.

Item	Detail
File	dictionary_generator.py
Function	generate_dictionary()
Input	names, years, symbols lists
Output	Set of ~42 password candidates
Techniques	Leet-speak, capitalization, year append, symbol append

Module 2 — Hash Extractor

Detects the running OS and extracts password hashes from the actual system — Linux /etc/shadow or Windows user accounts.

Item	Detail
File	hash_extractor.py
Linux Source	/etc/shadow file (requires root)
Windows Source	wmic useraccount (shows SID and status)
Algorithms ID'd	MD5 (1), SHA-256 (5), SHA-512 (6), NTLM (32 hex chars)

Module 3 — Brute Force Simulator

Tries every word in the wordlist against the target password using MD5 hashing. Reports attempts, time taken, and result.

Item	Detail
File	brute_force.py
Function	brute_force(target)
Charset	a-z + 0-9 (lowercase + digits)
Max Length	Up to 5 characters
Hash Method	MD5 (hashlib)

Module 4 — Password Analyzer

Evaluates a password against 5 criteria and calculates Shannon entropy to measure unpredictability.

Check	Criteria
Length	8 or more characters
Lowercase	Contains a-z
Uppercase	Contains A-Z
Digits	Contains 0-9
Symbols	Contains !@#$% etc.
Score	0-5 (one point per check passed)

Module 5 — Report Generator

Compiles all results into a formatted text report and saves it as audit_report.txt.

Section	Content
Dictionary	Total words generated, sample list
Brute Force	Cracked password (if found)
Analysis	Password, score, entropy, rating
Recommendations	4 actionable security tips

Security Notes & Recommendations

Hash Algorithm Security

Algorithm	Rating	Notes
MD5	**WEAK**	Cracked in seconds, no salt
NTLM	**WEAK**	No salt, GPU-accelerated cracking
SHA-256	**MEDIUM**	Better but aging, use with salt
SHA-512	**GOOD**	Salted, slow enough for most uses
Bcrypt	**STRONG**	Adaptive cost, best for passwords

Password Policy Recommendations

- Minimum 12 characters (16+ for admin accounts)
- Mix uppercase, lowercase, digits, and symbols
- Never use names, birthdays, or keyboard patterns
- Use a different password for every account
- Enable Two-Factor Authentication (2FA) everywhere
- Use a password manager (e.g. Bitwarden, 1Password)
- Block passwords found in known breach databases

Ethical & Legal Disclaimer

IMPORTANT — Authorized Use Only

This toolkit is designed exclusively for:

- Authorized penetration testing with written permission
- Educational cybersecurity lab environments
- Security auditing of systems you own

Unauthorized use is illegal under CFAA, UK Computer Misuse Act, and equivalent laws.

How to Run

Step 1 — Place all 6 files in the same folder:

- main.py
- dictionary_generator.py
- hash_extractor.py
- brute_force.py
- password_analyzer.py
- report_generator.py

Step 2 — Open terminal in that folder and run:

```
python main.py
```

Step 3 — Follow the prompts:

- Enter the password you want to analyze (e.g. Admin@2024)
- Enter the short password you want to crack (e.g. ab1)

Step 4 — View the output:

- Results print to the terminal step by step
- Final report is saved as audit_report.txt in the same folder